

MATRIMORPHOSIS – 2026

ASSIGNMENT 2

FOUNDATIONS OF PROGRAMMING

Name : Shahnaz Shafreen S
College : Bannari Amman Institute of Technology
Department : Computer Science and Engineering
Batch : 2023 - 2027
Email ID : shafreenshahnaz@gmail.com
Date : 22-08-2026

1. What is a transpiler? Can you name a popular programming language that uses a transpiler?

- A transpiler (source-to-source compiler) is a program that converts source code written in one programming language into source code of another language. It is useful when the original language provides features that are not directly supported by the target environment. The converted code can then be compiled or executed by the target platform.
- Example: TypeScript is transpiled into JavaScript before it is executed by a browser or JavaScript runtime.

2. What are the executable container formats used in Windows 11 and MacOS?

- Operating systems use specific file formats to store executable programs and related information.
- Windows 11: Common executable formats include .exe for executable programs and .dll for dynamic libraries.
- macOS: Applications are commonly distributed as .app bundles, while the actual executable files inside them generally use the Mach-O format.
- These formats allow the operating system to load and execute programs correctly.

3. Do you know of any notation used to describe the syntax of programming languages? Try to express an email address in one of these notations.

- Backus–Naur Form (BNF) is a formal notation used to describe the syntax or grammar of programming languages. It represents how different elements of a language can be combined to form valid expressions.
- A simplified email address can be represented as:
 <email> ::= <username> "@" <domain>
 <username> ::= <letters-or-digits>
 <domain> ::= <name> "." <extension>
- For example: user@example.com

- Here, user is the username, example is the domain name, and com is the extension.

4. Do you think a program without functions will be faster than a program for the same logic implemented with functions? Why?

- Not necessarily. A program written without functions may avoid a small amount of function-call overhead, but this does not mean it will always be faster. Functions make programs easier to organize, reuse, test, and maintain.
- Modern compilers can also optimize function calls in many situations. Therefore, the performance difference is usually small and depends on the language, compiler, and type of function being used.

5. Can errors in call stack or stack frame lead to program errors like wrong functionality or program crashes?

- Yes. Errors involving the call stack or stack frames can lead to incorrect functionality, unexpected behavior, or program crashes. Each function call creates a stack frame containing information such as parameters, local variables, and the return address.
- For example, if a recursive function keeps calling itself without a proper stopping condition, too many stack frames are created. Eventually, the available stack memory can be exhausted, resulting in a stack overflow.

6. Do you think security related vulnerabilities could be associated with call stacks? Can you think of an example for this?

- Yes. The call stack can be associated with certain security vulnerabilities because it stores important control information such as return addresses and local variables.
- A common example is a stack-based buffer overflow. If a program stores more data in a stack buffer than the allocated space can hold, the extra data may overwrite nearby stack information. In vulnerable programs, this can potentially allow an attacker to modify the program's execution.

7. How will the set up of the call stack work for a recursive function call?

- When a recursive function calls itself, a new stack frame is created for every call. Each frame stores the function's parameters, local variables, and return information.
- For example, if a function calls itself three times, the call stack will contain multiple frames:

Recursive Call 3

Recursive Call 2

Recursive Call 1

Original Call

- When the base condition is reached, the function returns. The stack frames are then removed one by one in the Last-In, First-Out (LIFO) order.

8. Can a call stack grow indefinitely? What happens if a function is called recursively too many times?

- No. A call stack cannot grow indefinitely because the program has a limited amount of stack memory.
- If a recursive function continues calling itself without reaching a base condition, a new stack frame is created for every call. Eventually, the stack memory becomes full and the program produces a stack overflow. Depending on the programming language and environment, this may cause an exception, program termination, or crash.

9. What does it mean to say that functions or expressions can have “side effects”?

- A side effect occurs when a function or expression changes something outside the value it directly produces. This can include changing a variable, modifying an array, writing to a file, printing output, or updating a database.
- For example:

`count = count + 1`

changes the value of count, so it has a side effect.

- A function that only calculates a value and returns it without changing anything outside itself is generally considered to have no side effects.

10. How can one stop the execution of a loop before it is completed? Illustrate the difference between 'break' and 'continue' with an example.

- The break and continue statements can change the normal execution of a loop.

Statement	Purpose
break	Completely terminates the loop
continue	Skips the current iteration and proceeds to the next iteration

- Example of break:

```
FOR i ← 1 TO 5
  IF i = 3
    BREAK
  PRINT i
END FOR
Output: 1 2
```

- Example of continue:

```
FOR i ← 1 TO 5
  IF i = 3
    CONTINUE
  PRINT i
END FOR
Output: 1 2 4 5
```

11. Write the pseudo code to find the sum of the cubes of the first 9 positive integers.

- The first nine positive integers are 1 to 9. We calculate the cube of each number and add all the cubes.

```
START
    sum ← 0
    FOR i ← 1 TO 9
        cube ← i × i × i
        sum ← sum + cube
    END FOR
    PRINT sum
END
```

The final result is: 2025

- Therefore, the sum of the cubes of the first 9 positive integers is 2025.

12. Write the pseudo code to check if a given 4-digit square number will yield another square number by adding 1 to each digit.

- The program first accepts a four-digit number, separates its digits, adds 1 to each digit, and forms a new number. It then checks whether the new number is a perfect square.

```
START
    INPUT n
    Extract the four digits of n
    Add 1 to each digit
    Form the new number
    Calculate the square root of the new number
    IF square root is an integer
        PRINT "It is a square number"
    ELSE
        PRINT "It is not a square number"
    END IF
END
```

➤ Example:

Input: 2025

After +1: 3136

56×56 : 3136

Since $3136 = 56^2$, the resulting number is a square number.